

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación

Carrera de Ingeniería de Software

TA-IND-02

Análisis de Consistencia y Replicación en Bases de Datos Distribuidas

Caso de estudio: AGLS – TiendaTech

Estudiante: José Alejandro Lozano Morales

Equipo PFC: AGLS – TiendaTech

Asignatura: Aplicaciones Distribuidas (ISR-701)

Unidad: 3 – Bases de Datos Distribuidas

Docente: Prof. Guerrero Ulloa G.C.

Período: 2026–2027

Fecha: 12 de julio de 2026

Abstract

This report analyzes the consistency and replication model actually implemented in the AGLS–TiendaTech Final Course Project, a hardware e-commerce platform with a component compatibility assistant. Inspection of the source code and deployment configuration shows a single Spring Boot application connected to one PostgreSQL instance through one data source, with the data and stored routines concentrated in the `public` schema. Consequently, the current system has neither physical replicas nor an implemented schema-per-service separation. Its observable guarantees correspond to PostgreSQL’s local transactional consistency, while its exact behavior under concurrent operations depends on the isolation level, locking strategy, and stored functions whose definitions are not included in the repository. The implemented model is compared with two alternatives: primary–follower replication with read replicas and automatic failover, and a database-per-service architecture coordinated through the Saga pattern. The analysis concludes that the current approach is reasonable for the academic stage because of its simplicity and local atomicity, but it is not optimal for production traffic due to its single point of failure. An incremental evolution toward primary–follower replication is recommended, preserving strong guarantees for inventory, orders, and payment operations.

Resumen

El presente informe analiza el modelo de consistencia y replicación realmente implementado en el PFC AGLS–TiendaTech, una plataforma de comercio electrónico de hardware con asistente de compatibilidad. La inspección del código y del despliegue evidencia una aplicación Spring Boot conectada a una única instancia PostgreSQL, con una sola fuente de datos y uso generalizado del esquema `public`; por tanto, aún no existen réplicas físicas ni separación efectiva por esquemas. El sistema obtiene consistencia transaccional local del motor, pero sus garantías exactas frente a operaciones concurrentes dependen del aislamiento, los bloqueos y las funciones almacenadas, cuya definición no está incluida en el repositorio. Se contrasta este modelo con dos alternativas: replicación primario–seguidor con réplica de lectura y failover, y base de datos por servicio con coordinación mediante Saga. Se concluye que el modelo actual es razonable para la fase académica por su simplicidad y atomicidad local, pero no es óptimo para un despliegue real debido al punto único de fallo. Se recomienda una evolución incremental hacia primario–seguidor, manteniendo consistencia fuerte en inventario, pedidos y pagos.

Índice

Abstract	1
Resumen	1
1. Introducción	3
2. Marco teórico	3
2.1. Espectro de modelos de consistencia	3
2.2. CAP y PACELC	4
2.3. Estrategias de replicación	4
2.4. Separación lógica y distribución física	4
3. Análisis del modelo implementado en TiendaTech	5
3.1. Evidencia obtenida del repositorio	5
3.2. Garantías de consistencia observables	5
3.3. Relación con el dominio y lectura CAP/PACELC	6
4. Evaluación de alternativas	7
4.1. Alternativa A: primario–seguidor con failover	7
4.2. Alternativa B: base de datos por servicio y Saga	7
5. Discusión crítica	7
6. Conclusiones	9
7. Declaración de uso de IA generativa	9

1. Introducción

TiendaTech es una plataforma nacional de comercio electrónico especializada en componentes informáticos. Su diferenciador funcional es un asistente que valida la compatibilidad entre procesadores, placas base, memorias, tarjetas gráficas, almacenamiento y fuentes de poder antes de que el cliente finalice una compra. El dominio combina consultas intensivas de catálogo con operaciones críticas de inventario, carrito, pedidos, pagos y facturación. Una lectura obsoleta en el catálogo puede ser tolerable durante un intervalo breve; en cambio, una lectura obsoleta del inventario durante el checkout puede provocar sobreventa, cobros sin producto disponible o pedidos imposibles de cumplir.

La documentación inicial del PFC propone seis dominios: Usuarios y Seguridad, Productos, Inventario, Pedidos, Ventas, y Órdenes y Proveedores. Como evolución arquitectónica, el equipo ha planteado que cada dominio utilice un esquema propio dentro de una única instancia PostgreSQL. Sin embargo, la consigna del TA-IND-02 exige estudiar el modelo *realmente implementado*, no únicamente el plan futuro. Por esta razón, el análisis se fundamenta en evidencia del repositorio: `docker-compose.yml`, `application.properties`, las entidades JPA, los servicios y las consultas SQL presentes en el código.

La pregunta orientadora es: **¿el modelo de consistencia elegido es óptimo para el dominio de TiendaTech?** Para responderla se distinguen tres niveles que no deben confundirse: la implementación actual, la evolución lógica por esquemas y las posibles arquitecturas con réplicas físicas. El objetivo es caracterizar las garantías observables, evaluar sus compromisos bajo CAP y PACELC, y comparar dos alternativas mediante criterios de consistencia, disponibilidad, latencia, tolerancia a particiones y complejidad operativa.

2. Marco teórico

2.1. Espectro de modelos de consistencia

La consistencia distribuida representa un espectro de garantías y no una propiedad binaria [1]. La *linealizabilidad* exige que cada operación parezca ocurrir instantáneamente entre su invocación y su respuesta, respetando el orden del tiempo real. La *consistencia secuencial* conserva un orden total común para todos los procesos, pero no exige que dicho orden coincida con el tiempo real. La *consistencia causal* preserva el orden de las operaciones causalmente relacionadas, mientras permite órdenes diferentes para operaciones concurrentes e independientes. Las garantías de sesión, como *read-your-writes*, aseguran que un cliente observe sus propias modificaciones. Finalmente, la *consistencia eventual* garantiza convergencia en ausencia de nuevas escrituras, sin prometer que una lectura inmediata obtenga el valor más reciente.

En términos relativos, el orden general de mayor a menor fuerza semántica es linealizabilidad, consistencia secuencial, consistencia causal, garantías de sesión como *read-your-writes* y consistencia eventual. Cuanto más fuerte es el modelo, mayor coordinación se requiere para ordenar y hacer visibles las operaciones, lo que normalmente incrementa la latencia y puede reducir la disponibilidad durante fallos o particiones; los modelos más débiles admiten respuestas con menor coordinación y mayor disponibilidad, a cambio de posibles lecturas obsoletas o

divergencias temporales.

Estas definiciones se aplican principalmente cuando existen varias copias de un dato. En una base de datos con una sola copia no existe divergencia entre réplicas ni retraso de propagación. No obstante, aún es necesario analizar el aislamiento entre transacciones concurrentes. PostgreSQL ofrece atomicidad, consistencia, aislamiento y durabilidad (ACID), pero el nivel de aislamiento determina qué anomalías pueden observarse. Una transacción ejecutada bajo `READ COMMITTED` no equivale automáticamente a una ejecución serializable ni permite afirmar que toda la aplicación sea linealizable.

2.2. CAP y PACELC

El teorema CAP formaliza que, cuando se produce una partición de red, un sistema replicado no puede garantizar simultáneamente disponibilidad para todas las solicitudes y una vista consistente de los datos [1]. La formulación no significa que un sistema elija permanentemente dos letras de tres; describe la decisión que debe tomar durante una partición.

PACELC amplía el análisis: si existe una partición, se elige entre disponibilidad y consistencia; de lo contrario, en operación normal, se elige entre latencia y consistencia. Una revisión abierta reciente confirma que este compromiso surge de la coordinación y sincronización necesarias entre réplicas, las cuales agregan latencia a las garantías fuertes [1]. En una instancia única, CAP no se manifiesta como un conflicto entre réplicas: ante la pérdida de acceso al único nodo, el servicio queda indisponible. PACELC tampoco presenta todavía un intercambio de latencia por coordinación entre copias, aunque sí existen costos de bloqueo y contención entre transacciones.

2.3. Estrategias de replicación

En la replicación primario–seguidor, un nodo acepta las escrituras y propaga sus cambios a uno o más seguidores. La propagación síncrona reduce el riesgo de pérdida y puede sostener garantías fuertes, pero añade latencia a cada confirmación. La propagación asíncrona reduce la latencia de escritura, aunque las lecturas dirigidas a seguidores pueden ser obsoletas. Las propuestas recientes intentan reducir este compromiso mediante nuevas topologías y paralelismo sin renunciar a la consistencia fuerte [2].

En la replicación multilíder, varios nodos aceptan escrituras y requieren resolución de conflictos. En los sistemas sin líder, las lecturas y escrituras se realizan sobre subconjuntos de réplicas. Los quóruns permiten obtener consistencia fuerte y tolerancia a fallos cuando se coordinan adecuadamente; QuorumX, por ejemplo, combina replicación de logs y reproducción eficiente para cargas OLTP [3]. Una propuesta abierta de 2025 emplea votación binaria sobre una organización lógica en cuadrícula para disminuir el costo de comunicación sin abandonar la consistencia fuerte [4]. Estas estrategias aumentan la disponibilidad, pero también la complejidad de operación, monitoreo, recuperación y pruebas.

2.4. Separación lógica y distribución física

Un esquema de PostgreSQL es un espacio de nombres dentro de una misma instancia. Separar tablas en `productos`, `inventario` o `ventas` mejora la propiedad lógica y permite

asignar permisos, pero no genera nodos independientes ni copias de los datos. Tampoco constituye, por sí solo, replicación o tolerancia a particiones.

Debe distinguirse además entre separación por dominio y fragmentación vertical clásica. La fragmentación vertical divide las columnas de una misma relación conservando una clave que permita reconstruirla. Agrupar tablas completas por dominio corresponde mejor al patrón *schema-per-service*. La partición de una tabla de ventas por rangos de fecha sí puede considerarse fragmentación horizontal lógica, pero mientras todos los fragmentos permanezcan en el mismo motor no existe distribución física.

3. Análisis del modelo implementado en TiendaTech

3.1. Evidencia obtenida del repositorio

El archivo `docker-compose.yml` declara solamente dos contenedores: una aplicación y una instancia `postgres:17`. Existe un único volumen persistente, `postgres_data`, y no aparecen nodos primarios, seguidores, balanceadores o servicios de failover. La aplicación configura una única conexión mediante `spring.datasource.url`, `spring.datasource.username` y `spring.datasource.password`. Además, Hibernate fija `public` como esquema predeterminado.

La evidencia del código coincide con esa configuración. Las consultas invocan objetos como `public.usuario`, `public.producto`, `public.movimiento_inventario`, `public.f_carrito_agregar` y `public.f_checkout_generar_orden_json`. No se encontraron los seis esquemas propuestos en la documentación, configuraciones de múltiples fuentes de datos, enrutamiento de lecturas, réplicas o parámetros de PostgreSQL como `primary_conninfo`, `standby.signal` o `synchronous_standby_names`.

También se observa un solo proyecto Maven, un solo artefacto Spring Boot y un solo proceso backend. Las clases denominadas `ProductoService`, `CarritoService` o `PaymentService` son servicios internos de la misma aplicación, no microservicios desplegados de forma independiente. En consecuencia, la implementación actual es un monolito modular con una base centralizada.

La Figura 1 muestra la diferencia entre el estado verificable y la intención del equipo. Esta distinción evita atribuir al sistema una topología distribuida inexistente.

3.2. Garantías de consistencia observables

El modelo implementado obtiene consistencia transaccional local de PostgreSQL. Una escritura confirmada y una lectura posterior dirigida a la misma instancia no sufren retraso de replicación. Varias operaciones utilizan `@Transactional`, lo que permite agrupar modificaciones dentro de una unidad atómica. Sin embargo, no se encontró una selección explícita de `SERIALIZABLE`, `REPEATABLE_READ` o bloqueos pesimistas y optimistas. Tampoco aparecen anotaciones `@Version`, `@Lock` o consultas `FOR UPDATE` en el código Java.

La operación de checkout invoca la función `public.f_checkout_generar_orden_json`. Al ser una única sentencia enviada a PostgreSQL, su ejecución ocurre dentro de una transacción del motor; no obstante, el repositorio no contiene la definición SQL de la función. Por ello, no puede demostrarse desde el código disponible si el checkout bloquea el inventario, valida el

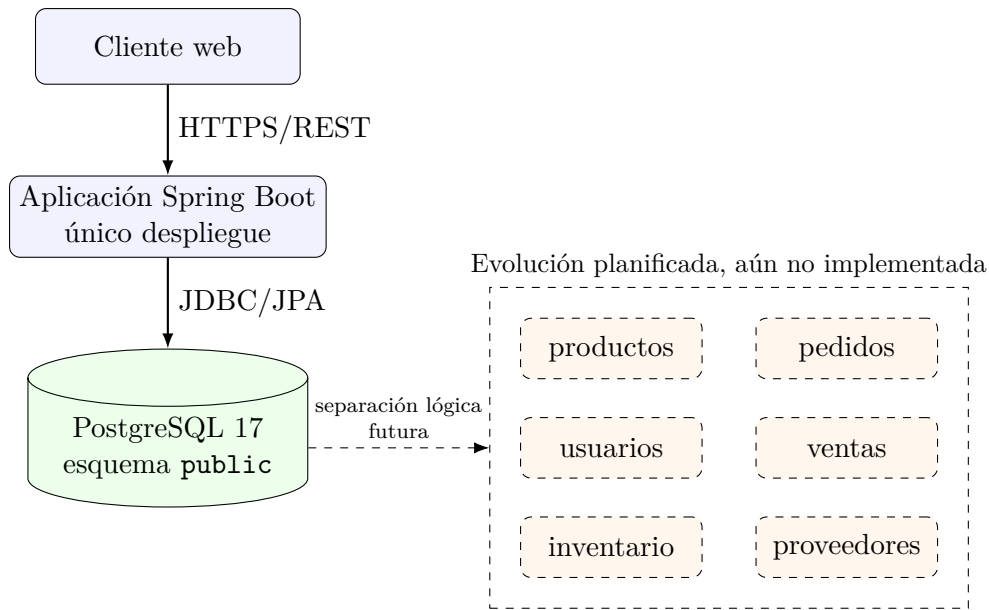


Figura 1: Topología propia basada en la evidencia del repositorio: instancia única y esquema `public`; a la derecha se representa la separación lógica planificada.

stock y lo descuenta mediante una actualización condicional, o si puede producir sobreventa frente a dos compras concurrentes.

La conclusión técnicamente prudente es que el sistema no implementa un modelo de consistencia entre réplicas. Implementa garantías ACID locales, cuya fortaleza efectiva depende del aislamiento predeterminado y de procedimientos almacenados no auditables desde el repositorio. Etiquetar toda la aplicación como linealizable sería una afirmación sin evidencia suficiente.

3.3. Relación con el dominio y lectura CAP/PACELC

El modelo centralizado simplifica el caso crítico de negocio: inventario, pedido, venta y factura pueden modificarse dentro del mismo motor sin una transacción distribuida. Esto reduce latencia y evita coordinar múltiples bases. Para una fase académica y una carga pequeña, la simplicidad tiene valor porque facilita pruebas y recuperación.

El costo es la disponibilidad. PostgreSQL constituye un punto único de fallo: si el contenedor, el host o el volumen quedan inaccesibles, los módulos dejan de operar simultáneamente. El sistema no es tolerante a particiones y no puede ofrecer servicio desde una segunda copia. En términos de PACELC, todavía no existe el intercambio normal entre latencia y consistencia derivado de coordinar réplicas; sí existe contención, pues catálogo, reportes y checkout compiten por conexiones, CPU, memoria y almacenamiento.

Los requisitos tampoco son homogéneos. Inventario, confirmación de pedidos, pagos y facturación requieren datos actuales y operaciones atómicas. Catálogo, imágenes, productos populares y recomendaciones toleran mejor un retraso acotado. Esta diferencia favorece una arquitectura híbrida futura: las operaciones críticas deben permanecer en el primario, mientras consultas no críticas pueden escalarse mediante seguidores de lectura o cachés controladas.

4. Evaluación de alternativas

4.1. Alternativa A: primario–seguidor con failover

La primera alternativa conserva una sola fuente autorizada para escrituras y añade uno o más seguidores mediante streaming replication. El checkout, inventario, pagos y pedidos se ejecutan en el primario. Las consultas tolerantes a obsolescencia, como catálogo o reportes, pueden dirigirse a un seguidor. El sistema verificable GroveKV demuestra que una arquitectura primario–respaldo puede ejecutar lecturas sobre réplicas mediante arrendamientos, recuperarse de fallos y aumentar su rendimiento al incorporar nodos de respaldo [5]; por ello, el enrutamiento debe decidirse por operación y garantía requerida, no globalmente.

Una réplica asíncrona reduce el costo de las escrituras, pero puede devolver valores atrasados. No debe utilizarse para verificar stock inmediatamente antes de vender. Una réplica síncrona disminuye la ventana de pérdida, aunque incrementa la latencia. Para resolver el punto único de fallo no basta una réplica de lectura: se requiere promoción automática, monitoreo y un mecanismo de redirección. Esta alternativa es incremental y mantiene la ventaja de las transacciones locales.

4.2. Alternativa B: base de datos por servicio y Saga

La segunda alternativa separa físicamente los dominios. Cada microservicio es propietario de su base y se comunica mediante API o eventos. Una venta que afecte inventario, pedidos y facturación deja de ser una transacción ACID única; se transforma en una Saga con pasos locales y acciones compensatorias. Si el cobro se confirma pero falla el descuento de inventario, el sistema debe reintentar, reservar previamente el producto o ejecutar una devolución.

La ventaja es el aislamiento: una carga intensa de catálogo no utiliza necesariamente los mismos recursos que pagos. También permite escalar y desplegar cada dominio de forma independiente. La desventaja es la consistencia eventual entre servicios, la dificultad de observabilidad y la necesidad de idempotencia, colas, reintentos y compensaciones. Para la fase actual, esta complejidad excede los beneficios inmediatos.

La Tabla 1 muestra que la alternativa A ofrece el mejor equilibrio inmediato. Las arquitecturas recientes también demuestran que es posible combinar replicación, consistencia fuerte y alto rendimiento mediante diseños especializados [6], pero su complejidad no se justifica todavía en TiendaTech. La literatura reciente identifica los modelos causales y eventuales como alternativas capaces de equilibrar ordenamiento, disponibilidad y rendimiento [1]; en TiendaTech resultan más apropiados para datos tolerantes a retraso que para la confirmación de inventario y cobro.

5. Discusión crítica

El modelo implementado no es óptimo para el dominio completo de TiendaTech, aunque sí es razonable para su etapa académica. Su principal ventaja es que concentra las operaciones críticas en un motor transaccional. Esto evita, por ahora, la complejidad de las transacciones distribuidas. Su principal defecto es que las metas declaradas de alta disponibilidad y tolerancia

Cuadro 1: Comparación del modelo implementado con dos alternativas.

Criterio	Actual: instancia única	A: primario-seguidor	B: BD por servicio + Saga
Consistencia	ACID local; sin divergencia entre copias; aislamiento no explicitado	Fuerte en el primario; posible <i>staleness</i> en seguidores asíncronos	ACID dentro de cada servicio; eventual entre servicios
Disponibilidad	Baja ante fallo de la instancia	Alta si existe promoción y redirección automáticas	Alta por dominio; fallos parciales posibles
Latencia	Baja, sin coordinación remota; posible contención	Mayor en replicación síncrona; lecturas escalables	Baja localmente, pero mayor en flujos coordinados
Particiones	No toleradas; pérdida de acceso implica indisponibilidad	Puede priorizar consistencia o lecturas disponibles según configuración	Aislamiento parcial; depende del bus y políticas de cada servicio
Complejidad	Baja	Moderada: réplica, monitoreo, failover y enrutamiento	Alta: eventos, idempotencia, Saga y compensaciones
Adecuación	Fase académica y tráfico reducido	Mejor evolución a corto y mediano plazo	Evolución futura si la escala justifica el costo

a fallos no están respaldadas por la topología: una instancia única no ofrece continuidad ante la caída del nodo.

La evolución por esquemas mejora la organización, pero no corrige ese defecto. Crear seis esquemas dentro del mismo PostgreSQL no añade copias, no distribuye carga entre servidores y no proporciona failover. Además, si los servicios acceden directamente a tablas de otros esquemas para compartir una transacción, conservan atomicidad pero aumentan el acoplamiento. Si se comunican únicamente por REST, respetan mejor los límites de servicio, pero una transacción local ya no cubre el proceso completo.

La recomendación es adoptar una política diferenciada. Inventario, pagos, pedidos y facturación deben leer y escribir en el primario con transacciones explícitas y control de concurrencia. El checkout debe verificarse mediante la definición SQL de `f_checkout_generar_orden_json`; idealmente debe bloquear la fila de inventario o ejecutar una actualización condicional que solo descuenta unidades cuando exista stock suficiente. El catálogo, los reportes y las recomendaciones pueden dirigirse a una réplica asíncrona con retraso monitoreado.

Antes de incorporar replicación debe cerrarse la brecha de reproducibilidad del repositorio: faltan migraciones o scripts SQL con tablas, restricciones, funciones y procedimientos. Sin esos artefactos no es posible auditar integralmente la consistencia, reconstruir la base o verificar el comportamiento concurrente. También deben definirse métricas: retraso máximo de réplica, objetivo de recuperación, pérdida de datos tolerable, latencia P95 del checkout y tasa de conflictos de stock.

En síntesis, la postura adoptada es que la instancia única debe mantenerse durante la fase académica, documentándola honestamente como arquitectura centralizada. La primera evolución debe ser primario-seguidor con failover, no una separación inmediata en seis bases. Esta decisión preserva la atomicidad local y mejora la disponibilidad con un costo operativo moderado. Una arquitectura por servicio y Saga queda como alternativa futura si el crecimiento y la independencia de equipos justifican su complejidad.

6. Conclusiones

1. La implementación inspeccionada consta de una aplicación Spring Boot y una sola instancia PostgreSQL. No existen microservicios desplegados de manera independiente, réplicas físicas ni configuración de propagación de datos.
2. Los datos y procedimientos utilizados por el código se encuentran en el esquema `public`. La separación en seis esquemas es una decisión planificada, no una característica implementada; además, representa aislamiento lógico, no replicación ni distribución física.
3. El modelo de consistencia observable corresponde a garantías transaccionales locales de PostgreSQL. No hay evidencia suficiente para clasificar toda la aplicación como linealizable o serializable, pues no se configura un aislamiento explícito y faltan las definiciones de las funciones almacenadas.
4. La instancia única simplifica la atomicidad de inventario, venta, pedido y factura, pero constituye un punto único de fallo. En consecuencia, el modelo es adecuado para la fase académica, pero no es óptimo para tráfico real ni cumple por sí solo los objetivos de alta disponibilidad.
5. La alternativa recomendada es una evolución primario–seguidor con failover automático y enrutamiento selectivo: operaciones críticas en el primario y consultas tolerantes a obsolescencia en seguidores. La base por servicio con Saga se reserva para una etapa de mayor escala.
6. Como mitigación inmediata, el equipo debe versionar los scripts SQL, documentar el aislamiento, verificar el control concurrente del stock y definir métricas de recuperación, retraso de réplica y latencia del checkout.

7. Declaración de uso de IA generativa

La redacción y la revisión inicial de la documentación y del código del PFC, realizadas con el propósito de obtener la información necesaria para esta tarea, fueron desarrolladas por el autor. Posteriormente, se utilizó ChatGPT, de OpenAI, como herramienta de apoyo para pulir la claridad y la coherencia de la redacción, organizar la estructura del documento en LaTeX y revisar la presentación del resumen, el marco teórico, el análisis técnico, la tabla comparativa, la discusión y las conclusiones. Las fuentes académicas empleadas fueron buscadas y seleccionadas inicialmente por el autor; la inteligencia artificial se utilizó como apoyo para comprobar sus datos bibliográficos, la correspondencia de los DOI y su pertinencia respecto del tema de consistencia y replicación. La interpretación del PFC, la postura crítica adoptada y la responsabilidad por el contenido final corresponden al autor.

Referencias

- [1] J. Ahmed, A. Karpenko, O. Tarasyuk, A. Gorbenko y A. Sheikh-Akbari, “Consistency Issue and Related Trade-Offs in Distributed Replicated Systems and Databases: A Review,” *Radioelectronic and Computer Systems*, n.º 2, págs. 171-179, 2023. DOI: [10.32620/reks.2023.2.14](https://doi.org/10.32620/reks.2023.2.14). dirección: <https://doi.org/10.32620/reks.2023.2.14>.
- [2] F. F. Altukhaim, A. M. Mostafa y A. E. Youssef, “Multidirectional Replication for Supporting Strong Consistency, Low Latency, and High Throughput,” *Alexandria Engineering Journal*, vol. 61, págs. 11 485-11 510, 2022. DOI: [10.1016/j.aej.2022.05.005](https://doi.org/10.1016/j.aej.2022.05.005). dirección: <https://doi.org/10.1016/j.aej.2022.05.005>.
- [3] D. Wang, P. Cai, W. Qian y A. Zhou, “Efficient and Stable Quorum-Based Log Replication and Replay for Modern Cluster-Databases,” *Frontiers of Computer Science*, vol. 16, n.º 5, pág. 165 612, 2022. DOI: [10.1007/s11704-020-0210-y](https://doi.org/10.1007/s11704-020-0210-y). dirección: <https://doi.org/10.1007/s11704-020-0210-y>.
- [4] A. A. C. Fauzi, N. Ashafiq, A. Mat, S. A. M. Nasir y A. Noraziah, “Re-Engineering Grid-Based Quorum Replication into Binary Vote Assignment on Cloud: A Scalable Approach for Strong Consistency in Cloud Databases,” *International Journal of Advanced Computer Science and Applications*, vol. 16, n.º 9, págs. 653-659, 2025. DOI: [10.14569/IJACSA.2025.0160962](https://doi.org/10.14569/IJACSA.2025.0160962). dirección: <https://doi.org/10.14569/IJACSA.2025.0160962>.
- [5] U. Sharma, R. Jung, J. Tassarotti, M. F. Kaashoek y N. Zeldovich, “Grove: A Separation-Logic Library for Verifying Distributed Systems,” en *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, ACM, 2023, págs. 113-129. DOI: [10.3929/ethz-b-000641983](https://doi.org/10.3929/ethz-b-000641983). dirección: <https://doi.org/10.3929/ethz-b-000641983>.
- [6] A. Krechowicz, S. Deniziak y G. Łukawski, “Consistent, Highly Throughput and Space Scalable Distributed Architecture for Layered NoSQL Data Store,” *Scientific Reports*, vol. 15, pág. 19 172, 2025. DOI: [10.1038/s41598-025-03755-5](https://doi.org/10.1038/s41598-025-03755-5). dirección: <https://doi.org/10.1038/s41598-025-03755-5>.